

PROJECT REQUIREMENT DOCUMENT (PRD)

Small-Scale Distribution Billing & Product Management Web App

Version 1.0 | September 2026

Document purpose: Implementation-ready requirements for a local, low-effort distribution billing web application.

Core rule: MRP is displayed for retailer reference; distributor rate is the actual billed rate; invoice totals are calculated only from distributor rate.

1. Project Overview

Purpose

Build a lightweight local web application for a small distribution business. The app will allow the business owner/operator to maintain products and variants, manage MRP and distributor selling rates, create bills quickly through an add-to-cart workflow, generate professional PDF invoices, print them, and manage historical bills—without needing to edit source code for routine business operations.

Primary business model

The business purchases/distributes FMCG-type products to shopkeepers/retailers. A product may have many pack sizes/variants (for example 250g, 500g, 1kg, 5kg). The shopkeeper generally sells to the end customer based on the product MRP. The distributor bill must therefore show both MRP and the distributor's actual selling rate, but the invoice total must be calculated only from the distributor rate.

Core principle

After deployment, normal business operations must be possible entirely through the app UI. Adding products, editing product names/variants, changing MRP, changing distributor rates, managing shops/customers, creating bills, viewing history, regenerating PDFs, and managing business/invoice settings must not require source-code changes.

2. Goals & Success Criteria

Goals

1) Create a bill in roughly one minute for a normal order. 2) Make price changes from the UI. 3) Generate a professional printable PDF. 4) Preserve historical bill accuracy even after prices change. 5) Keep the system simple enough for non-technical daily use. 6) Avoid unnecessary cloud infrastructure for the first version.

Success criteria

An operator can search/select products, choose quantities, review a cart, and generate a correct PDF invoice without touching code. A product's current MRP/rate can be changed from the admin UI. Old bills retain the exact prices used when they were generated. PDF totals, quantities, and line amounts reconcile exactly.

3. Scope

In scope - MVP

Product/variant management; MRP and distributor-rate management; shop/customer management; cart-based billing; quantity editing; bill preview; invoice numbering; professional PDF generation; print-friendly output; bill history; PDF regeneration; business profile/settings; basic validation; local SQLite persistence; backup/export; responsive desktop/tablet UI; clear error messages.

Optional / Phase 2

Stock/inventory tracking; payment status; credit/udhaar ledger; GST/tax calculations; discounts; returns/credit notes; sales analytics; Excel/CSV export; WhatsApp sharing; cloud sync; multi-user login; barcode scanning; purchase management; supplier management.

Out of scope for MVP

Online payments, public customer ordering, multi-branch synchronization, complex accounting/ERP functionality, automatic tax compliance, external e-commerce integrations, and dependence on a paid cloud backend.

4. Recommended Technical Approach

Stack

Frontend/UI: Streamlit. Application logic: Python. Database: SQLite. PDF: ReportLab. Local runtime: localhost. Packaging/run: standard Python virtual environment with a simple start command. The architecture should keep business logic separate from UI and PDF generation even though the application is small.

Why this stack

Low development effort, fast UI development, excellent fit for a local internal tool, easy Python-based PDF generation, and no need for a separate React frontend plus Node backend for the MVP.

Architecture

UI → application/service layer → SQLite database. Bill generation reads a validated cart, creates a bill transaction with price snapshots, then passes immutable bill data to the PDF generator. Generated PDFs are stored in a predictable invoices/output folder. Routine configuration is stored in the database/settings rather than hardcoded.

5. User Roles

Primary role - Business Owner/Operator

Full access to billing, products, pricing, shops, history, settings, backup/restore, and PDF generation.

Future role - Staff

Optional future role with billing access but restricted pricing/settings access. Authentication is not required for MVP unless the deployment environment requires it.

6. Functional Requirements - Product Management

Product model

A product represents the general product/brand, while each sellable pack size/variant is a separate billable item. Example: Surf Excel is the product; 250g, 500g, 1kg and 5kg are variants/SKUs.

Required fields

Product/brand name; variant/pack size; MRP; distributor selling rate; active/inactive status. Optional: SKU/code, category, unit, notes, barcode, display order.

Operations

Add product/variant; edit; deactivate; reactivate; search; filter; duplicate/copy an existing variant; delete only when safe/appropriate. Prefer deactivation over hard deletion once an item has appeared in a bill.

Validation

Name cannot be blank. Variant should be identifiable. MRP must be non-negative and normally distributor rate should not exceed MRP unless an explicit business rule is later enabled. Numeric values must accept decimal currency if required. Duplicate product+variant combinations should be prevented or clearly warned.

7. Functional Requirements - Pricing & MRP

Two-price model

Every sellable variant must support: MRP = reference/display price for the retailer; Distributor Rate = actual price charged by this business. Bill line amount = quantity × Distributor Rate. MRP must NOT be included in the invoice grand total.

Dynamic pricing

Prices must be editable from the app. No source-code editing should be required. The current product record contains the current MRP and distributor rate.

Historical price snapshot

When a bill is finalized, store the product name, variant, MRP, distributor rate, quantity and line amount inside the bill/bill-item record. Future price edits must not alter old bills.

Optional future price history

A later version may maintain a full price-change audit trail with effective date/time and previous/new values.

8. Functional Requirements - Shop/Customer Management

Shop fields

Shop name; owner/contact person (optional); phone (optional); address (optional); GSTIN (optional); notes (optional); active/inactive status.

Operations

Add shop, edit shop, search shop, select shop during billing, and optionally create a new shop from the billing screen without leaving the bill workflow.

Fast billing requirement

The operator should be able to select an existing shop quickly. A 'Walk-in/Other' or generic shop option may be available if the business wants bills without maintaining a customer record.

9. Functional Requirements - Billing / Cart

Billing workflow

1) Start New Bill. 2) Select shop/customer. 3) Search product or browse categories. 4) Select variant. 5) Add to cart. 6) Enter/change quantity. 7) Review cart. 8) Generate/Finalize Bill. 9) Create PDF. 10) Open/print/save PDF.

Cart line

Each line must show product name, variant, quantity, MRP, distributor rate, and calculated amount. Quantity controls should support quick +/- changes and direct numeric entry.

Totals

Line amount = quantity × distributor rate. Subtotal/grand total is the sum of line amounts. MRP values are informational and must not be summed into the amount due unless a future feature explicitly introduces an MRP-total comparison report.

Bill edit behavior

Before finalization, cart contents can be changed. After finalization, the bill should be treated as immutable. If correction is needed, use a future edit/cancel/reissue mechanism rather than silently modifying a finalized invoice.

Rounding

Use a consistent currency precision rule, preferably 2 decimal places internally and in PDF output, while allowing whole-rupee display where appropriate. The implementation must avoid floating-point money errors by using Decimal or integer minor units.

10. Invoice / PDF Requirements

Required PDF content

Business name/logo; business address/contact; invoice number; invoice date/time; shop/customer name and details if available; item table; product/variant; quantity; MRP; distributor rate; line amount; grand total; optional notes/footer; page number if multi-page.

Critical pricing presentation

The PDF must clearly distinguish MRP from the distributor rate. Recommended columns: Item/Variant | Qty | MRP | Rate | Amount. The Amount column is based on Rate only.

Example calculation

Surf Excel 500g: Qty 10, MRP ■75, Distributor Rate ■61, Amount ■610. The invoice total contribution is ■610, not ■750.

Professional appearance

Clean A4 invoice layout, consistent typography, aligned currency columns, clear total box, business header, adequate spacing, no clipped text, and print-friendly black/white readability. Logo should be optional. The PDF should remain readable if printed in grayscale.

PDF lifecycle

On finalization, create a uniquely named PDF such as INV-000123.pdf. Store it locally. Bill history should provide an Open/View PDF or Regenerate PDF action. Regeneration must use the stored historical snapshot, not today's product prices.

Invoice numbering

Sequential invoice number with configurable prefix, e.g. INV-000001. The next number must persist across app restarts. Number generation must avoid duplicates.

11. Business Settings

Editable settings

Business/legal name; display name; address; phone; email (optional); GSTIN (optional); logo; invoice prefix; starting invoice number (only before use or through a controlled setting); currency symbol; default invoice footer/notes; PDF output directory; optional signature text.

No-code requirement

All ordinary settings must be editable through the Settings UI. The user should not have to open Python files or configuration files for normal business changes.

12. Bill History

History list

Show invoice number, date, shop, total amount, and status. Provide search/filter by invoice number, shop, and date range.

Actions

Open/view PDF; regenerate PDF from stored snapshot; view bill details; optionally duplicate a previous bill into a new draft; export later if required.

Data integrity

Deleting a product or changing its current price must never delete or alter finalized bills.

13. Dashboard / Home

Recommended dashboard

Quick actions: New Bill, Products, Shops, Bill History, Settings. Optional summary cards: Today's bill count, today's sales total, current active products.

Speed

The New Bill action should be prominent because daily billing is the primary use case.

14. Backup & Data Safety

Local backup

Provide an in-app 'Backup Database' action that creates a timestamped copy of the SQLite database. Optionally provide a ZIP containing database + generated invoices.

Restore

Provide a controlled restore mechanism or, at minimum for MVP, a clearly documented way to replace the local database after backup. A safer UI-based restore can be Phase 2.

Data persistence

Restarting the application must not lose products, prices, shops, bills, settings, or invoice numbering.

No accidental deletion

Destructive actions should require confirmation. Prefer deactivate/archive for products and shops that appear in historical records.

15. Database Requirements

Core tables

products; shops/customers; bills; bill_items; settings. Optional later: price_history, categories, users, payments, stock.

Bill data model

Bill stores invoice number, timestamp, shop reference/name snapshot, totals, notes, status. Bill item stores product reference where useful plus immutable snapshots of product name, variant, MRP, rate, quantity and amount.

Relationships

One bill has many bill items. One shop can have many bills. One product/variant can appear in many bill items.

16. UI / UX Requirements

Design

Simple business-first interface. Avoid unnecessary animations and clutter. Use clear buttons and large enough quantity controls for fast operation.

Search

Product search should match product name and variant. Shop search should match shop name/phone.

Keyboard friendliness

Where Streamlit permits, optimize for quick keyboard use: search first, easy quantity editing, clear primary action, and minimal navigation.

Responsive behavior

Desktop/laptop is the primary target; tablet-friendly layout is desirable. Mobile support is optional and should not compromise desktop billing speed.

Language

UI may be English initially. Currency should support ■ / INR. Labels should be understandable to a non-technical operator.

17. Validation & Error Handling

Examples

Do not allow blank product names; invalid/negative numeric values; zero/negative quantity; missing required shop details where a shop is required; duplicate invoice numbers; incomplete bill finalization; corrupt/missing PDF output folder. Show actionable messages rather than Python tracebacks to the business user.

Recovery

If PDF generation fails after a bill is finalized, the finalized bill data must remain saved and the app should offer a retry/regenerate action. It must not create duplicate invoice numbers just because PDF generation was retried.

18. Security / Reliability

MVP security

Because this is a local internal app, complex authentication is optional. If exposed beyond localhost, authentication and access controls become mandatory.

Reliability

Database writes should be transactional for bill finalization. The app should handle restarts safely. Invoice numbering must be atomic enough to avoid duplicate numbers.

Privacy

Customer/shop data is business data and should remain local unless cloud sync is intentionally added later.

19. Performance Requirements

Billing speed

Normal product search and cart updates should feel immediate on the local machine. PDF generation should normally complete within a few seconds for a standard one-page bill.

Scale target

MVP should comfortably handle at least hundreds to a few thousand product variants and thousands of historical bills on a local SQLite database without requiring migration to a server database.

20. Acceptance Criteria / Definition of Done

Functional acceptance

A new product can be added entirely from UI. Its MRP and distributor rate can be edited from UI. A shop can be added from UI. A bill can be created with multiple products and quantities. Amounts use distributor rate only. MRP is shown but excluded from total. Finalized bill stores price snapshots. PDF contains all required invoice fields. PDF can be opened and printed. Old PDF/history remains correct after price changes. Invoice numbers persist and remain unique. App restart preserves all data. Backup can be created.

Agent handoff test

The implementation agent must provide a seeded demo dataset, a repeatable setup/run procedure, and a short manual test checklist covering: add product; edit MRP/rate; create shop; create bill; verify calculations; generate PDF; change product price; verify old bill remains unchanged; restart app; verify data persists; create backup.

21. Recommended Project Structure

Structure

app.py / main entry; database.py / persistence; models or schema module; services/billing_service.py; services/product_service.py; services/shop_service.py; pdf_generator.py; UI/page modules; database/billing.db; invoices/; backups/; assets/logo; requirements.txt; README.md. Exact file organization may vary, but responsibilities should remain separated.

Configuration rule

Business data and settings must live in the database or controlled configuration, not hardcoded in UI code. The app should work after cloning/setup without requiring manual source edits for normal use.

22. Future Enhancements

Phase 2 candidates

Stock quantities and low-stock alerts; purchase cost and margin reports; customer credit/udhaar; payment tracking; returns; discounts; GST/tax; Excel export; WhatsApp PDF sharing; barcode scanner; cloud backup; multi-device access; staff accounts; analytics dashboard.

Important future design note

Do not implement Phase 2 complexity unless required. However, the MVP database and services should be structured so these features can be added without rewriting billing logic.

23. Sample End-to-End Scenario

Scenario

Product: Surf Excel 500g. MRP ■75. Distributor Rate ■61. Shopkeeper orders 10 units. Cart shows Qty 10, MRP ■75, Rate ■61, Amount ■610. Add Vim 500ml: MRP ■85, Rate ■62, Qty 8 → ■496. Grand Total = ■1,106. PDF shows both MRP and Rate for each item but total payable is ■1,106. Later Surf Excel rate changes from ■61 to ■64. A new bill uses ■64, while the old bill remains ■61.

24. Agent Implementation Instructions

Must follow

Treat this document as the source of truth for MVP behavior. Do not introduce unnecessary technologies. Do not hardcode products/prices. Do not make price changes require code edits. Do not calculate invoice total from MRP. Do not mutate historical bill prices after product edits. Keep PDF generation deterministic and print-friendly.

Deliverables

Working local web app; database schema; seeded sample data; professional PDF template; requirements file; README with setup/run/backup instructions; basic automated tests for billing calculations and invoice numbering; manual acceptance checklist; sample generated PDF.

Handoff expectation

The final application should be usable by a non-technical business operator. If a setting can reasonably be changed during normal business operations, expose it in the UI rather than asking the operator to edit source code.